

Sampling & Fourier Series

Practice Problems with Python Solution Code

This set covers the core topics typically tested in a sampling / Fourier series unit: computing Fourier coefficients (analytically and via the DFT), the sampling theorem and Nyquist rate, aliasing, signal reconstruction (sinc interpolation), Parseval's theorem, spectral leakage and windowing, zero-padding, non-uniform sampling, and quantization noise. Each problem includes a runnable Python solution using NumPy (and Matplotlib for plots).

Part 1 - Fourier Series Coefficients

Question 1: Compute Fourier Series Coefficients from Samples

Problem. Given a periodic signal $x(t) = 3 + 2\cos(2\pi t) + \sin(4\pi t)$ with period $T = 1$, sample it at $N = 16$ points over one period and compute the Fourier series coefficients using the DFT. Compare with the known analytical coefficients.

Solution code:

```
import numpy as np

N = 16          # number of samples
T = 1.0        # period
t = np.arange(N) * T / N

# Signal:  $x(t) = 3 + 2\cos(2\pi t) + \sin(4\pi t)$ 
x = 3 + 2*np.cos(2*np.pi*t) + np.sin(4*np.pi*t)

# DFT gives Fourier series coefficients (scaled)
X = np.fft.fft(x) / N

print("DFT coefficients (first few):")
for k in range(5):
    print(f"c_{k} = {X[k]:.4f}")

# Analytical:  $c_0 = 3$ ,  $c_1 = 1$  (from cos term),  $c_{-1} = 1$ 
#  $\sin(4\pi t) \rightarrow k = 2$ ,  $c_2$  has imaginary part  $-0.5j$ 
print("Expected: c0=3, c1=1 (real), c2 = -0.5j")
```

Key idea: The DFT of N uniformly spaced samples over one period equals the Fourier series coefficients (scaled by N), as long as the signal is band-limited to fewer than $N/2$ harmonics.

Question 2: Numerical Integration for Fourier Coefficients

Problem. Compute the complex exponential Fourier series coefficients c_k (for $k = -3..3$) of a periodic triangle wave using numerical integration (trapezoidal rule) instead of the DFT, and compare against the DFT-based estimate.

Solution code:

```
import numpy as np

T = 2*np.pi
def triangle(t):
    t = np.mod(t, T)
```

```

        return np.where(t < np.pi, (2/np.pi)*t - 1, 3 - (2/np.pi)*t)

# Fine grid for numerical integration
M = 2000
t = np.linspace(0, T, M, endpoint=False)
x = triangle(t)
dt = T / M

def fourier_coeff(k):
    integrand = x * np.exp(-1j*2*np.pi*k*t/T)
    return np.sum(integrand) * dt / T

print("Coefficients via numerical integration:")
for k in range(-3, 4):
    print(f"c_{k} = {fourier_coeff(k):.4f}")

# Compare with DFT on a coarser uniform grid
N = 32
tn = np.arange(N) * T / N
xn = triangle(tn)
X = np.fft.fft(xn) / N
X_shifted = np.fft.fftshift(X)
print("\nDFT-based estimate (should match closely):", X_shifted[N//2-3:N//2+4])

```

Key idea: $c_k = (1/T) \int_0^T x(t) e^{-j 2 \pi k t / T} dt$ over one period. Numerical integration and the DFT should agree for a smooth, well-sampled signal.

Part 2 - Sampling Theorem & Aliasing

Question 3: Nyquist Rate Check and Aliasing Demonstration

Problem. For $x(t) = \cos(2\pi \cdot 50t) + \cos(2\pi \cdot 120t)$, determine the Nyquist rate, then sample at (a) above and (b) below the Nyquist rate and observe aliasing.

Solution code:

```

import numpy as np

f1, f2 = 50, 120
f_max = max(f1, f2)
nyquist_rate = 2 * f_max
print(f"Nyquist rate = {nyquist_rate} Hz")

def sample_signal(fs, duration=0.05):
    t = np.arange(0, duration, 1/fs)
    x = np.cos(2*np.pi*f1*t) + np.cos(2*np.pi*f2*t)
    return t, x

# (a) Proper sampling
fs_good = 300 # > 240 Hz
t1, x1 = sample_signal(fs_good)

# (b) Undersampling -> aliasing
fs_bad = 150 # < 240 Hz, 120 Hz component will alias
t2, x2 = sample_signal(fs_bad)

f_alias = abs(f2 - fs_bad * round(f2/fs_bad))
print(f"120 Hz component aliases to: {f_alias} Hz when fs = {fs_bad} Hz")

```

Key idea: Nyquist rate = $2 * f_{\max}$. Sampling below it causes higher frequencies to 'fold back' and appear as lower, aliased frequencies.

Question 4: Aliasing - Identify and Verify via FFT Peak Detection

Problem. Sample a 40 Hz sine wave at 30 Hz (below Nyquist). Identify the aliased frequency analytically, then confirm it numerically by finding the FFT peak.

Solution code:

```
import numpy as np

f_signal = 40    # Hz
fs = 30          # sampling rate (below Nyquist!)
duration = 2
t = np.arange(0, duration, 1/fs)
x = np.sin(2*np.pi*f_signal*t)

X = np.fft.fft(x)
freqs = np.fft.fftfreq(len(t), 1/fs)

pos_mask = freqs >= 0
peak_idx = np.argmax(np.abs(X[pos_mask]))
observed_freq = freqs[pos_mask][peak_idx]

alias_freq = abs(f_signal - fs * round(f_signal/fs))

print(f"True frequency: {f_signal} Hz")
print(f"Sampling rate: {fs} Hz")
print(f"Theoretical alias: {alias_freq} Hz")
print(f"Observed (from FFT): {observed_freq:.2f} Hz")
```

Key idea: Alias frequency formula: $f_{\text{alias}} = |f - fs * \text{round}(f / fs)|$.

Question 5: Minimum Sampling Rate for a Band-Limited Signal

Problem. A signal has a spectrum that is nonzero only for $200 \text{ Hz} \leq |f| \leq 500 \text{ Hz}$ (a bandpass signal). Estimate the minimum uniform sampling rate needed to avoid aliasing using the standard baseband Nyquist criterion, and verify by simulating a multi-tone signal within that band.

Solution code:

```
import numpy as np

f_low, f_high = 200, 500
f_max = f_high

# Baseband (conservative) Nyquist criterion
fs_min_baseband = 2 * f_max
print(f"Conservative minimum sampling rate: {fs_min_baseband} Hz")

# Build a test signal with several tones inside the band
fs = fs_min_baseband + 50    # sample comfortably above minimum
duration = 0.05
t = np.arange(0, duration, 1/fs)
tones = [220, 350, 480]
x = sum(np.cos(2*np.pi*f*t) for f in tones)

X = np.fft.rfft(x)
freqs = np.fft.rfftfreq(len(t), 1/fs)
peaks = freqs[np.argsort(np.abs(X))[-len(tones):]]
print("Detected tone frequencies:", np.sort(peaks))
```

Key idea: For a general (non-bandpass-optimized) system, the safe rule is $fs > 2*f_{\text{max}}$ of the highest frequency present, even if the signal energy starts above 0 Hz. (Bandpass sampling can sometimes use a lower rate, but that is a more advanced case.)

Part 3 - Reconstruction from Samples

Question 6: Reconstruct a Signal from Truncated Fourier Series (Gibbs Phenomenon)

Problem. Given a square wave, compute its Fourier series up to N harmonics and reconstruct it. Show how the reconstruction improves with more harmonics, and observe the overshoot near discontinuities (Gibbs phenomenon).

Solution code:

```
import numpy as np
import matplotlib.pyplot as plt

def square_wave_fourier(t, N_harmonics, T=2*np.pi):
    x_approx = np.zeros_like(t)
    for k in range(1, N_harmonics+1, 2): # odd harmonics only
        x_approx += (4/(np.pi*k)) * np.sin(k * (2*np.pi/T) * t)
    return x_approx

t = np.linspace(0, 2*2*np.pi, 1000)

plt.figure(figsize=(10, 6))
for N in [1, 5, 15, 50]:
    y = square_wave_fourier(t, N)
    plt.plot(t, y, label=f'N={N} harmonics')
plt.legend()
plt.title("Square Wave Approximation via Fourier Series")
plt.xlabel("t")
plt.savefig("square_wave.png")
plt.show()
```

Key idea: Gibbs phenomenon: near a jump discontinuity, truncated Fourier series always overshoot by about 9% of the jump size, no matter how many harmonics are used.

Question 7: Ideal Reconstruction of a Band-Limited Signal (Sinc Interpolation)

Problem. Sample a band-limited signal above the Nyquist rate, then reconstruct the continuous-time signal from the samples using sinc (Whittaker-Shannon) interpolation. Compare the reconstructed signal to the original.

Solution code:

```
import numpy as np
import matplotlib.pyplot as plt

f0 = 5          # signal frequency (Hz)
fs = 50         # sampling rate, well above Nyquist (10 Hz)
Ts = 1/fs

n = np.arange(-20, 21)
t_samples = n * Ts
x_samples = np.cos(2*np.pi*f0*t_samples)

def sinc_reconstruct(t_cont, t_s, x_s, Ts):
    x_rec = np.zeros_like(t_cont)
    for tn, xn in zip(t_s, x_s):
        x_rec += xn * np.sinc((t_cont - tn) / Ts)
    return x_rec

t_cont = np.linspace(-0.5, 0.5, 2000)
```

```

x_rec = sinc_reconstruct(t_cont, t_samples, x_samples, Ts)
x_true = np.cos(2*np.pi*f0*t_cont)

error = np.max(np.abs(x_rec - x_true))
print(f"Max reconstruction error: {error:.6f}")

plt.plot(t_cont, x_true, label="Original")
plt.plot(t_cont, x_rec, '--', label="Sinc-reconstructed")
plt.stem(t_samples, x_samples, linefmt='g-', markerfmt='go', basefmt=' ')
plt.legend()
plt.savefig("sinc_reconstruction.png")

```

Key idea: $\text{np.sinc}(x)$ computes $\sin(\pi x)/(\pi x)$. Whittaker-Shannon interpolation $x(t) = \sum x[n] * \text{sinc}((t - nTs)/Ts)$ perfectly reconstructs a signal sampled above the Nyquist rate (in theory, with infinite samples).

Part 4 - Spectral Analysis Details

Question 8: Verify Parseval's Theorem Using Sampled Data

Problem. For a sampled periodic signal, verify that average power computed in the time domain equals the sum of squared magnitudes of the Fourier coefficients.

Solution code:

```

import numpy as np

N = 32
t = np.arange(N) / N
x = 2*np.cos(2*np.pi*3*t) + 1.5*np.sin(2*np.pi*7*t) + 1

# Time-domain average power
power_time = np.mean(np.abs(x)**2)

# Frequency-domain (Parseval via DFT)
X = np.fft.fft(x) / N
power_freq = np.sum(np.abs(X)**2)

print(f"Time-domain power:      {power_time:.6f}")
print(f"Frequency-domain power: {power_freq:.6f}")
print(f"Match: {np.isclose(power_time, power_freq)}")

```

Key idea: Parseval's theorem: average power in the time domain equals the sum of squared magnitudes of the Fourier coefficients in the frequency domain.

Question 9: Spectral Leakage and the Effect of Windowing

Problem. Sample a sinusoid whose frequency does NOT correspond exactly to an FFT bin. Observe spectral leakage in the rectangular-windowed FFT, then apply a Hann window and compare the spectra.

Solution code:

```

import numpy as np

fs = 100
N = 64
t = np.arange(N) / fs
f0 = 12.5 # not an integer multiple of fs/N -> leakage

x = np.sin(2*np.pi*f0*t)

# Rectangular window (no windowing)
X_rect = np.fft.rfft(x)

```

```

freqs = np.fft.rfftfreq(N, 1/fs)

# Hann window
w = np.hanning(N)
X_hann = np.fft.rfft(x * w)

print("Peak bin (rectangular):", freqs[np.argmax(np.abs(X_rect))])
print("Peak bin (Hann):", freqs[np.argmax(np.abs(X_hann))])

# Compare spread of energy around the peak (leakage indicator)
def spread(X):
    mag = np.abs(X)
    return np.sum(mag > 0.1 * mag.max())

print("Bins above 10% of peak (rectangular):", spread(X_rect))
print("Bins above 10% of peak (Hann):", spread(X_hann))

```

Key idea: When a signal's frequency falls between DFT bins, energy 'leaks' into neighboring bins. Windowing (Hann, Hamming, etc.) reduces leakage at the cost of a wider main lobe.

Question 10: Zero-Padding and Frequency Resolution

Problem. Show that zero-padding a sampled signal before taking the FFT increases the number of interpolated frequency points (finer plotting resolution) but does NOT increase the true frequency resolution (which is set by the observation time).

Solution code:

```

import numpy as np

fs = 100
N = 32
t = np.arange(N) / fs
x = np.cos(2*np.pi*10*t) + np.cos(2*np.pi*11.5*t) # two close tones

def spectrum(x_data, n_fft):
    X = np.fft.rfft(x_data, n=n_fft)
    freqs = np.fft.rfftfreq(n_fft, 1/fs)
    return freqs, np.abs(X)

# Original N-point FFT
f1, m1 = spectrum(x, N)
# Zero-padded to 8x length
f2, m2 = spectrum(x, 8*N)

print(f"True frequency resolution (1/T): {fs/N:.3f} Hz")
print(f"Bin spacing without padding: {f1[1]-f1[0]:.3f} Hz")
print(f"Bin spacing with 8x zero-padding: {f2[1]-f2[0]:.3f} Hz")
print("Zero-padding gives a smoother-looking spectrum, but cannot separate")
print("tones closer than ~fs/N without more actual samples.")

```

Key idea: True frequency resolution = $fs / N = 1 / (\text{total observation time})$. Zero-padding only interpolates the DFT; it does not add new information.

Part 5 - Practical & Advanced Topics

Question 11: Non-Uniform Sampling and Interpolation

Problem. Sample a signal at random (non-uniform) time points, then reconstruct it on a uniform grid using linear and cubic interpolation, and compare the errors.

Solution code:

```
import numpy as np
from scipy.interpolate import interp1d

np.random.seed(0)
f0 = 3
t_true = np.linspace(0, 1, 1000)
x_true = np.sin(2*np.pi*f0*t_true)

# Non-uniform samples
n_samples = 40
t_nu = np.sort(np.random.uniform(0, 1, n_samples))
x_nu = np.sin(2*np.pi*f0*t_nu)

lin_interp = interp1d(t_nu, x_nu, kind='linear', fill_value="extrapolate")
cub_interp = interp1d(t_nu, x_nu, kind='cubic', fill_value="extrapolate")

x_lin = lin_interp(t_true)
x_cub = cub_interp(t_true)

err_lin = np.mean((x_lin - x_true)**2)
err_cub = np.mean((x_cub - x_true)**2)

print(f"Linear interpolation MSE: {err_lin:.6f}")
print(f"Cubic interpolation MSE: {err_cub:.6f}")
```

Key idea: Non-uniform sampling breaks the standard FFT/Nyquist framework; reconstruction typically relies on interpolation methods instead.

Question 12: Quantization Noise After Sampling

Problem. Sample a sinusoid, then quantize the samples to B bits. Compute the resulting quantization noise power and compare it to the theoretical estimate ($\Delta^2 / 12$), and compute the resulting SNR.

Solution code:

```
import numpy as np

fs = 1000
f0 = 50
duration = 0.1
t = np.arange(0, duration, 1/fs)
x = np.sin(2*np.pi*f0*t) # amplitude range [-1, 1]

def quantize(signal, bits, v_range=(-1, 1)):
    levels = 2**bits
    delta = (v_range[1] - v_range[0]) / levels
    q = np.round((signal - v_range[0]) / delta) * delta + v_range[0]
    return q, delta

for bits in [4, 8, 12]:
    xq, delta = quantize(x, bits)
    noise = xq - x
    noise_power_measured = np.mean(noise**2)
    noise_power_theory = delta**2 / 12
    signal_power = np.mean(x**2)
    snr_db = 10*np.log10(signal_power / noise_power_measured)
    print(f"{bits}-bit: measured noise power={noise_power_measured:.3e}, "
          f"theory={noise_power_theory:.3e}, SNR={snr_db:.2f} dB")
```

Key idea: Quantization noise power is approximately $\Delta^2/12$, where Δ is the step size. SNR improves by roughly 6 dB per additional bit.

Tip: Run each snippet independently (each imports what it needs). For questions using matplotlib, the `.savefig()` calls will write PNG files you can open to inspect the plots.